

INTERVIEW PREPARATION · END-TO-END UNDERSTANDING

Production API Wrapper

A plain-English study guide for the whole project

FastAPI · OpenAI Responses API · Redis
prod-api-wrapper

Study outside-in: problem > doors > guards > two jobs > Redis > failures > tests

Contents

1. The one-sentence story
2. The picture to keep in your head
3. How to study this project
4. Stage 0 — The business problem
5. Stage 1 — The six public doors
6. Stage 2 — How the app boots
7. Stage 3 — The three guards
8. Stage 4 — Synchronous path (wait for the answer)
9. Stage 5 — Background path, polling, and webhooks
10. Stage 6 — Errors, health, and fail-closed
11. Stage 7 — Redis as the shared brain
12. Stage 8 — Tests as a second teacher
13. Stage 9 — Built vs still on the old roadmap
14. File map
15. Whiteboard scripts
16. Five-day study plan
17. Interview questions and answers
18. Words you must not mix up

How to use this PDF: read one stage, open the listed files, then close the laptop and explain the stage in plain English. If you cannot explain it without looking, you do not own it yet.

1. The one-sentence story

This project is a **gatekeeper** between your internal application and OpenAI. Your app never talks to OpenAI. It talks to this wrapper. The wrapper holds the real OpenAI key, checks who is calling, limits how often they can call, stops accidental double billing, and either waits for the answer or says “here is a job ID, poll later.”

Interview opener: “This is not a chatbot. It is an internal production API that hides the provider, shrinks the contract, and adds the controls you need before you let many services call a paid model.”

If you can tell that story clearly, you already have the first two minutes of the interview. Everything else is detail that proves you actually built it.

What this project proves

- An internal application can use AI without holding the OpenAI API key.
- A small, explicit API contract is safer to operate than forwarding every upstream option.
- Redis can coordinate state shared by multiple API instances.
- Background work needs durable job state and verified event delivery.
- Reliable services make retries safe, failures predictable, and requests traceable.

2. The picture to keep in your head

```

Internal app
  | Bearer key + Idempotency-Key + optional X-Correlation-ID
  v
FastAPI wrapper
  | 1. Stamp a correlation ID
  | 2. Check the internal key
  | 3. Count this request in Redis (rate limit)
  | 4. Validate the small request body
  | 5. Claim "I already did this?" in Redis
  | 6. Call OpenAI (or skip if already done)
  |
+-----> OpenAI Responses API
  |
+-----> Redis
          rate-limit counters
          idempotency records
          job records
          processed webhook events

OpenAI later
  | signed webhook
  v
POST /webhooks/openai
  | verify signature, update job in Redis
  v
Internal app polls GET /v1/responses/{job_id}
  
```

The three actors and what each one is allowed to know.

Actor	Talks to	Holds	Must never see
Internal app	Only this wrapper	Wrapper API key	OpenAI API key, webhook secret, Redis internals
This wrapper	OpenAI + Redis	OpenAI key, webhook secret, wrapper key	Must not leak secrets in logs or error bodies
OpenAI	Webhook URL on this wrapper	The model work	Your wrapper bearer key (webhooks use a signature instead)

3. How to study this project

Do not start in random files. Do not start with `roadmap.md` — that is a build plan, not a teaching order. Study **outside-in**, like a request walking through a building.

- **Why** this service exists
- The **six public doors** (endpoints)
- What happens to **every** request (middleware)
- The two real jobs: **wait for the answer** vs **start a job**
- **Redis**: the shared notebook
- **Failures**: retries, errors, health
- **Tests**: how you prove it works
- What is still unfinished, so you do not oversell

You already have notes in the `Learning/` folder. Use those *after* you understand the story. This PDF is the teaching order.

4. Stage 0 — The business problem

Question you must answer: why not give every service the OpenAI key?

- Keys leak from frontends and from many repositories.
- Every team would invent their own retries, limits, and errors.
- OpenAI's API is huge; your company only needs a few fields.
- Background work needs a place to store “is this job done?”
- Networks retry; you must not pay twice for the same click.

Read

`Learning/README.md`, `roadmap.md` sections 1–2, and the first ten answers in `Learning/questions-and-answers.md`.

Interview line: “The wrapper is a security and operations boundary. The product talks to our API. We talk to OpenAI.”

You are done with this stage when...

You can explain the project to a non-engineer in under one minute, and then add the engineering reasons in a second minute.

5. Stage 1 — The six public doors

Question: what can a caller actually do? The wrapper publishes a small, stable contract. Unknown request fields are rejected. All bodies are JSON.

Door	Who is allowed	Success	What it means in English
POST /v1/responses	Internal bearer key + Idempotency-Key	200	Do this now and wait.
POST /v1/responses/background	Internal bearer key + Idempotency-Key	202	Start this, do not wait. Return a job id.
GET /v1/responses/{job_id}	Internal bearer key	202 or 200	Is that job done yet? Reads Redis only.
POST /webhooks/openai	OpenAI signature, not your key	200	OpenAI is telling us the job finished.
GET /health/live	Internal bearer key	200	Is the process alive? Never talks to Redis or OpenAI.
GET /health/ready	Internal bearer key	200 or 503	Can I do real work? Pings Redis.

The only fields a caller may send

Both create endpoints accept the same body. Extra fields are rejected (`extra='forbid'` on the Pydantic model).

Field	Required	Validation
input	Yes	Non-blank string, 1–50,000 characters.
instructions	No	If present: non-blank, at most 10,000 characters.
model	No	Must be in <code>OPENAI_ALLOWED_MODELS</code> . If omitted, <code>OPENAI_DEFAULT_MODEL</code> is used.
max_output_tokens	No	Integer from 1 to 16,384.
metadata	No	At most 16 string key/value pairs. Keys 1–64 chars, values at most 512.

The wrapper never accepts client-supplied API keys, tools, streaming options, callbacks, or arbitrary OpenAI fields. That is a design choice, not a missing feature.

Standard headers

Header	Who sends it	Rule
Authorization: Bearer ...	Internal app	Required on every door except the OpenAI webhook.
Idempotency-Key	Internal app	Required on both create endpoints. Makes retries safe.
X-Correlation-ID	Optional from client	Must match <code>^[A-Za-z0-9][A-Za-z0-9._-]{0,127}\$</code> . If missing, the wrapper creates one.

Read

Learning/api-design.md, app/schemas/responses.py, app/schemas/jobs.py.

Practice

Draw the six endpoints on paper. For each, write: who authenticates, success code, who calls it, and whether Redis or OpenAI is touched.

6. Stage 2 — How the app boots

`app/main.py` is the front door of the code. On startup (`lifespan`) the process:

- Loads settings from the environment (`load_settings`).
- Opens one shared Redis client.
- Opens one shared OpenAI client.
- Serves traffic.
- On shutdown, closes both clients (OpenAI first, then Redis).

Middleware order — this is an interview question

In code the middlewares are added as: rate limiting, then authentication, then correlation. Starlette wraps the last one first. So a real request hits:

```
correlation -> authentication -> rate limit -> route
```

- **Correlation first** so even a 401 still has an ID on the response.
- **Auth before rate limit** so random internet traffic does not burn Redis counters.
- **Webhooks skip auth and rate limit** because OpenAI cannot send your bearer key.

Configuration that refuses to start

`app/config.py` fails at boot if anything important is missing or invalid. Secrets are stored with `repr=False` so they do not print in errors.

Setting	Why it exists
<code>OPENAI_API_KEY</code>	The real provider key. Lives only in this service.
<code>OPENAI_WEBHOOK_SECRET</code>	Used to verify that a webhook really came from OpenAI.
<code>WRAPPER_API_KEY</code>	The internal key your own app sends.
<code>REDIS_URL</code>	Must be <code>redis://</code> or <code>rediss://</code> with a hostname.
<code>OPENAI_ALLOWED_MODELS</code>	Comma list. Callers cannot pick any model they want.
<code>OPENAI_DEFAULT_MODEL</code>	Must be on the allowlist.
<code>OPENAI_TIMEOUT_SECONDS</code>	How long we wait for OpenAI. Default 30.
<code>RATE_LIMIT_REQUESTS / WINDOW</code>	Shared fixed-window limit. Defaults 60 / 60.
<code>JOB_TTL_SECONDS</code>	How long a background job lives in Redis. Default 24 hours.
<code>IDEMPOTENCY_TTL_SECONDS</code>	How long a replay record lives. Default 24 hours.
<code>LOG_LEVEL</code>	CRITICAL, ERROR, WARNING, INFO, or DEBUG.

The OpenAI client is built with `max_retries=0`. That is deliberate: **you** retry in `retry_async`, the SDK does not. Otherwise you would double-retry and hide the policy.

Read

`app/main.py`, `app/config.py`, `app/clients/openai_client.py`,
`app/clients/redis_client.py`.

7. Stage 3 — The three guards

These three middlewares run on (almost) every request. Interviewers love this layer because it is where “toy API” becomes “production API.”

7.1 Correlation ID — a name tag, not a password

File: `app/middleware/correlation.py`

- Client may send X-Correlation-ID.
- If missing or invalid, the wrapper generates a UUID.
- Invalid format returns 400 `invalid_correlation_id`.
- The ID is always returned in the response header.
- It is attached to errors and to background job records.
- It does **not** authorize anyone. It only lets you find one request in logs.

7.2 Authentication — prove you are the internal app

File: `app/middleware/authentication.py`

- Header must be `Authorization: Bearer <WRAPPER_API_KEY>`.
- Compared with `hmac.compare_digest` so timing is harder to guess.
- The webhook path is skipped; it uses OpenAI’s signature instead.
- Failed auth never reaches Redis-backed business work or OpenAI.

7.3 Rate limiting — one shared counter

Files: `app/middleware/rate_limiting.py` and `app/services/rate_limit.py`.

- Fixed window in Redis: one counter per time window (`wrapper:rate-limit:{window_index}`).
- `INCR` and `EXPIRE NX` run in one transaction.
- Over the limit -> 429 plus `Retry-After`.
- Health and webhooks are exempt. Probes must still work. OpenAI must still deliver.
- If Redis is down, rate limiting fails closed with 503. Live still works.

Why Redis, not a Python variable? Two workers or two machines would each have their own counter. After a restart the count would be zero. Redis is the shared counter that survives process death.

Why must increment and expiry be atomic? If two first-in-window requests race, one might increment while neither sets expiry, leaving a counter that never resets.

You are done with this stage when you can say: “If Redis is down, rate limiting fails closed with 503. Live still works. Correlation still stamps every response.”

8. Stage 4 — Synchronous path (wait for the answer)

This is the core interview walkthrough. File: `app/api/responses.py` -> `create_response`.

Walk this until you can do it on a whiteboard with no notes.

- Require Idempotency-Key. Empty -> 422.
- Pick the model: request model or default. Must be on the allowlist -> otherwise 400 `unsupported_model`.
- Hash the validated request (`request_hash`). The hash includes the effective model so “same key, different model” is a conflict.
- Claim the key in Redis with `SET NX` and an `in_progress` record.
- First caller: you own the work.
- Same key + same body + done: return the saved answer. No second OpenAI call.
- Same key + same body + still running: 409 `idempotency_in_progress`.
- Same key + different body: 409 `idempotency_key_reused`.
- Call OpenAI through `retry_async`.
- If OpenAI fails for good: **delete** the claim so a later retry can try again.
- Shrink OpenAI’s fat object into your small response (`normalize_response`).
- Store the completed record in Redis.
- Return it as 200.

What you send to OpenAI

Only `input`, `model`, and the optional `instructions`, `max_output_tokens`, and `metadata`. Nothing else.

What you return to the caller

Field	Meaning
<code>id</code>	A wrapper id, <code>wrp_resp_...</code> , not OpenAI’s id.
<code>openai_response_id</code>	The provider id, kept for traceability.
<code>status</code>	Provider status, usually completed.
<code>model</code>	Which model actually ran.
<code>output_text</code>	The text the internal app needs.
<code>usage</code>	Token counts when the provider sent them.
<code>correlation_id</code>	The same name tag as the request.

Normalization lives in `app/services/responses.py`. The route owns the wrapper contract. The OpenAI SDK stays behind `app/clients/openai_client.py`. That split makes tests easy: fake the client, keep the contract.

Retries — only the flaky stuff

File: `app/services/retry.py`.

- At most 2 extra tries (3 attempts total).
- Retry only transient errors: connection errors, timeouts, provider 429, provider 5xx.
- Never retry auth failures, validation, or unsupported models.
- Delay is Retry-After if the provider sent one, otherwise exponential backoff plus jitter.
- Jitter exists so a provider outage does not create a synchronized retry storm.

Idempotency — the client sent this already

File: `app/services/idempotency.py`.

- The Redis key is `wrapper:idempotency:{sha256(key)}`, not the raw header. That keeps keys short and avoids odd characters in Redis.
- `SET NX EX` means “create only if missing, and auto-delete later.”
- On final provider failure the in-progress record is deleted so the next attempt can claim.
- If storing the completed result fails, the claim is deleted and the caller gets 503. Better to let them retry than to leave a stuck lock or a success the wrapper cannot replay.

Two words people mix up. Retry = this service tries OpenAI again after a blip. Idempotency = the client sent the same request again and you must not start a second billable job. Retries without idempotency can double-charge. Idempotency without retries still fails on a flaky network.

Read next

`app/services/idempotency.py`, `app/services/retry.py`, `app/services/responses.py`.

9. Stage 5 — Background path, polling, and webhooks

This is the hardest part of the project. Slow down. There is no Celery worker and no background thread doing the model work. OpenAI does the long work when you send `background=True`. You only store state and listen.

9.1 Create a background job

POST `/v1/responses/background` in `app/api/responses.py`.

- Same auth, validation, rate limit, and idempotency as the sync path.
- The idempotency hash includes `background: true` so a sync call and a background call with the same body are different work.
- Create a wrapper job id: `job_` plus a UUID4.
- Save the job in Redis as `pending` with an expiry.
- Call OpenAI with `background=True`.
- Save OpenAI's response id on the job and mark it `in_progress`.
- Also store a reverse lookup: OpenAI id -> your job id, so a webhook can find the job.
- Store the public 202 body in the idempotency record.

- Return 202 with `status_url`.

If OpenAI or Redis fails after the job was created, `_release_background` deletes the job and the idempotency claim so the client can retry cleanly.

9.2 Poll the job

GET `/v1/responses/{job_id}`.

- Rejects ids that are not `job_` plus a lowercase UUID4. That stops garbage and path tricks from hitting Redis oddly.
- Reads **only Redis**. Does not call OpenAI. Does not extend the TTL.
- Still running (pending / in_progress) -> 202.
- Finished / failed / cancelled / incomplete -> 200.
- Missing or expired -> 404 `job_not_found`. Same error on purpose, so people cannot fish for whether an old id once existed.

Job states

State	Meaning	Typical HTTP on GET
pending	Job saved, OpenAI not yet attached.	202
in_progress	OpenAI is working. We have their response id.	202
completed	Webhook (or retrieve) filled in the result.	200
failed	OpenAI reported failure.	200
cancelled	OpenAI reported cancellation.	200
incomplete	OpenAI stopped without a full result.	200
expired	Treated like missing. Public contract is 404.	404

9.3 Receive a signed webhook

Files: `app/api/webhooks.py` and `app/services/webhooks.py`.

- Require the three headers: `webhook-signature`, `webhook-timestamp`, `webhook-id`.
- Read the raw body. Cap at 1 MiB -> 413 if larger.
- Verify the signature **before** trusting anything (`openai.webhooks.unwrap`).
- Unknown event types return `{"received": true}` and stop. Do not fail OpenAI's delivery for events you do not use.
- Claim the event id in Redis with `SET NX` and a random owner token.
- Already processed -> 200, do nothing.
- Another worker is processing -> 503 so OpenAI retries.
- Look up the job by OpenAI response id.
- If the job is not there yet (webhook won the race) -> 503 so OpenAI retries.
- If the job is already terminal -> mark the event processed, do not overwrite.
- If the event is completed: retrieve the full response from OpenAI, then save text and usage.

- If retrieve does not yet show completed -> release the claim and 503.
- Finalize with a Lua script: only the owner token can finish; never overwrite a terminal job.

Events this wrapper cares about

OpenAI event	Job status written
response.completed	completed (after retrieve)
response.failed	failed
response.cancelled	cancelled
response.incomplete	incomplete

Why 503 instead of 200 when something is temporarily wrong

A 200 tells OpenAI “I got it, stop sending.” If Redis was down or the job was not stored yet, that event is lost. A 503 tells OpenAI “try again.” That is how you keep delivery honest without a queue of your own.

Owner tokens — why they exist

A worker claims an event as `processing:<token>`. If that worker dies, the claim expires and another worker can take over. Lua scripts check the token before release, mark-processed, or finalize. An expired worker cannot clobber a newer worker’s claim.

Interview trap: “Why no worker process?” Because OpenAI performs the model work. A second worker would only duplicate what webhooks already do, and you would still need Redis for shared state.

10. Stage 6 — Errors, health, and fail-closed

File: `app/errors.py`. Every error looks the same. Messages never include keys, prompts, raw OpenAI exceptions, or authorization headers.

```
{
  "error": {
    "code": "rate_limit_exceeded",
    "message": "Request limit exceeded.",
    "correlation_id": "request-123"
  }
}
```

HTTP	Code	When
400	<code>invalid_request</code> / <code>unsupported_model</code> / <code>invalid_correlation_id</code>	JSON is valid but violates this contract, or the model is not allowed, or the correlation header is malformed.
401	<code>authentication_failed</code> / <code>invalid_webhook_signature</code>	Bad wrapper key, or forged / unsigned webhook.
404	<code>job_not_found</code> (or <code>not_found</code>)	Unknown or expired job, or a missing route.
409	<code>idempotency_key_reused</code> / <code>idempotency_in_progress</code>	Same key, different body; or the first request is still running.

HTTP	Code	When
413	request_too_large	Webhook body over 1 MiB.
422	validation_error	Field type, presence, or size is wrong. Also missing idempotency key.
429	rate_limit_exceeded	This wrapper's Redis counter said stop. Includes Retry-After.
500	internal_error	Unexpected bug. Correlation middleware catches leftovers.
503	upstream_unavailable	Redis down, OpenAI down, OpenAI 429, connection error, or a webhook that must be retried.
504	upstream_timeout	OpenAI did not answer before OPENAI_TIMEOUT_SECONDS.

Live vs ready

File: `app/api/health.py`.

- **Live** = the process and HTTP server are up. It does not ping Redis or OpenAI. A platform should not kill the process just because Redis blinked.
- **Ready** = Redis answers PING. If not, stop sending this instance work. It can recover when Redis returns.
- Ready does **not** call OpenAI on every probe. That would be slow, billable, and a self-inflicted outage during a provider incident.

Interview line: “Live is 200 and ready is 503 means the process is up and Redis is not. Stop sending it traffic. Do not kill it yet.”

11. Stage 7 — Redis as the shared brain

Redis is required. Rate-limit counters, idempotency records, job state, and processed webhook events must be shared by all API instances and must survive a process restart. In-memory state cannot do that.

Key prefix	Purpose	Dies after
<code>wrapper:rate-limit:{window}</code>	Request counter for the current fixed window	The window length
<code>wrapper:idempotency:{sha256(key)}</code>	Claim + saved result for one client key	Default 24 hours
<code>wrapper:job:{job_id}</code>	The job JSON the client polls	Remaining job TTL
<code>wrapper:openai-response-job:{sha256(id)}</code>	OpenAI response id -> wrapper job id	Same remaining job TTL
<code>wrapper:webhook:event:{sha256(event_id)}</code>	Dedup + owner-token claim for one webhook	Processing TTL while working; 3 days once processed

Four Redis patterns you must be able to name

Pattern	Where	Plain English
SET NX EX	Idempotency claim, webhook event claim	Create this key only if it does not exist, and auto-delete later.

Pattern	Where	Plain English
Pipeline / transaction	Rate limit INCR+EXPIRE, job + reverse index	Two commands as one unit so a crash cannot leave half a write.
Lua script	Webhook release / mark / finalize	Only delete or update if I still own this token. Never overwrite a terminal job.
TTL	Every key	Nothing lives forever. An expired job is a 404.

Hashes appear on idempotency keys, OpenAI response ids, and webhook event ids so Redis keys stay a predictable length and do not contain raw caller input.

12. Stage 8 — Tests as a second teacher

You do not need to memorize tests. Use them as stories. `tests/conftest.py` fakes Redis and OpenAI. The suite is designed so CI never spends money and never needs a real OpenAI key.

Test file	Story it tells
<code>test_authentication.py</code>	No key / bad key never reaches OpenAI.
<code>test_config.py</code>	Bad environment refuses to boot. Secrets stay out of errors.
<code>test_health.py</code> / <code>test_readiness.py</code>	Live is not ready. Ready pings Redis.
<code>test_rate_limiting.py</code> / <code>test_rate_limit_service.py</code>	Shared Redis limit. Health and webhooks exempt. Atomic window.
<code>test_responses.py</code>	Sync happy path, allowlist, validation, extra fields rejected.
<code>test_retry.py</code> / <code>test_upstream_errors.py</code>	What is retried vs translated into wrapper errors.
<code>test_idempotency.py</code> / <code>test_idempotency_service.py</code>	Replay, conflict, in-progress, failed claim released.
<code>test_background_responses.py</code>	202, job created, OpenAI called with <code>background=True</code> .
<code>test_job_status.py</code> / <code>test_jobs_service.py</code>	Poll codes, unknown job, reverse index.
<code>test_webhooks.py</code> / <code>test_webhook_service.py</code>	Bad signature, duplicate, unknown event, owner token.
<code>test_openai_lifecycle.py</code>	Client is created and closed with the app lifespan.

Study method: pick one test name, predict the status code and whether OpenAI is called, then read the test. If your prediction is wrong, you found a gap.

13. Stage 9 — Built vs still on the old roadmap

Be honest in interviews. Claiming Docker and structured logging when they are not in the repo is worse than saying what you would add next.

Built and worth talking about

- Config that fails closed at boot
- Bearer auth and correlation IDs
- Redis clients, live, and ready
- Shared fixed-window rate limit

- Synchronous OpenAI Responses API with a small contract
- Bounded jittered retries and error translation
- Idempotency with Redis claims and replay
- Background jobs, status polling, signed webhooks
- Consistent error shape
- Automated tests with fakes, no live spend

Not finished as originally planned

- Rich structured JSON logs with named events (today: a simple line in correlation middleware).
- Docker / Docker Compose for API + Redis.
- A full root README and a portfolio demo script.

If asked “what would you add next?” say: structured logs with event names and correlation IDs, Docker Compose for local Redis + API, and per-client keys instead of one shared wrapper key.

14. File map

File	Job in one line
app/main.py	Start, stop, wire routes, register middleware and error handlers.
app/config.py	Read env, validate, refuse unsafe boot.
app/errors.py	One error shape. Special cases for model, idempotency, job 404.
app/api/responses.py	Sync create, background create, job poll.
app/api/webhooks.py	Verify OpenAI events and update jobs.
app/api/health.py	Live vs ready.
app/middleware/correlation.py	Name-tag every request.
app/middleware/authentication.py	Require the internal bearer key.
app/middleware/rate_limiting.py	Shared Redis limit, with exemptions.
app/services/idempotency.py	Claim, store, replay, delete.
app/services/retry.py	Bounded jittered retries of transient errors.
app/services/jobs.py	Redis job records and OpenAI-id reverse index.
app/services/webhooks.py	Event claim and Lua finalize.
app/services/rate_limit.py	Fixed-window INCR + EXPIRE NX.
app/services/responses.py	Shrink the provider payload.
app/schemas/responses.py	The public create request and sync response.
app/schemas/jobs.py	Job states and the public job body.
app/clients/*	SDK construction only. Routes do not build clients.

15. Whiteboard scripts

Practice these out loud. Time yourself. Two minutes each is enough if you stay concrete.

Walk me through a successful sync request.

Correlation stamps an ID. Auth checks the bearer key. Rate limit increments Redis. Pydantic validates the small body. The model is allowlisted. Idempotency SET NX claims the key. OpenAI is called through `retry_async`. The result is normalized, stored, and returned as 200 with the correlation header.

A client timed out and sent the same request again.

Same Idempotency-Key and same request hash. The record is completed, so we replay the stored result. OpenAI is not called. No second bill.

They reused the key with a different prompt.

The hash does not match. 409 `idempotency_key_reused`. The key is a fingerprint of one request, not a blank “do anything” token.

Two requests with the same key arrive at the same time.

Only one SET NX wins. The loser sees an `in_progress` record and gets 409 `idempotency_in_progress`. At most one OpenAI create happens.

Why is there no Celery or worker?

OpenAI runs the long job when we send `background=True`. We persist a Redis job, return 202, and later update that job from a verified webhook. A worker would duplicate that.

The webhook arrives before we saved the job.

Reverse lookup misses. We release the event claim and return 503. OpenAI retries. We do not ack early, because 200 would drop the event.

Why verify the webhook before parsing?

The signature covers the raw body and delivery headers. A forged request must not choose a job, trigger an OpenAI retrieve, or change shared Redis state.

Live is 200 but ready is 503. What is wrong?

The process is up. Redis is not answering PING. Stop sending this instance work. Do not kill the process yet; it can recover.

Why translate OpenAI errors instead of forwarding them?

Callers get a stable code and a correlation ID. We hide credentials, raw exceptions, and provider-shaped payloads that would couple every client to OpenAI’s error format.

What happens after 24 hours?

Job and idempotency TTLs expire. Polling an old job is 404. A new request with the same idempotency key is treated as new work. That is a cost/privacy trade-off, not an accident.

16. Five-day study plan

Each day: about 40 minutes reading the listed files, then 20 minutes speaking the story as if an interviewer is in the room. Do not skip the speaking part.

Day	Goal	You are done when
1	Story + six endpoints + boot + middleware	You can draw the architecture from memory, including middleware order.
2	Sync path + idempotency + retries	You can whiteboard Stage 4 without opening the file.
3	Background + poll + webhooks	You can explain owner tokens, reverse lookup, and why 503 exists.
4	Redis keys + error table + live/ready	You can list every Redis prefix and every status code.
5	Tests + Q&A; + “what I would add”	You can answer the questions in section 17 cold, and you do not oversell unfinished work.

If you only have one evening, do Day 2 and Day 3. Those two stories win the interview.

17. Interview questions and answers

These answers match the decisions in this repo. Learn them in your own words, not as a script.

1. Why put a wrapper in front of OpenAI?

To keep the OpenAI key in one place, give internal apps a small stable contract, and centralize auth, rate limits, retries, idempotency, and logs.

2. Why should a frontend never receive the OpenAI API key?

Anything shipped to a browser or a mobile app can be extracted. A leaked key is an open bill. The frontend talks to your backend; only this service talks to OpenAI.

3. Why expose only a limited set of fields?

A limited contract is easier to validate, document, test, and evolve. You add a field when there is a real internal use case instead of coupling every client to every upstream option.

4. Why reject unknown request fields?

So callers cannot silently depend on behavior you have not documented or tested. Typos fail at the boundary instead of being ignored.

5. Why enforce a model allowlist?

Cost, capability, and policy. Callers may only use configured models. The default must itself be on the list or the process will not start.

6. Why normalize provider responses?

The provider object is large and can change. Returning a stable shape keeps clients decoupled and gives you one place to control what is exposed.

7. Why is Redis required?

Rate-limit counters, idempotency records, job state, and processed webhook events must be shared across instances and survive a restart.

8. Why not an in-memory rate-limit counter?

Each worker would see a different count. A restart would wipe history. Two instances would allow twice the traffic.

9. Why are health routes exempt from rate limiting?

Probes must stay observable during traffic spikes. They should not consume the business request budget.

10. How should a caller handle 429?

Read Retry-After, wait at least that long, then retry with backoff. Do not tight-loop.

11. What is idempotency?

Doing the same intended action more than once has the same effect as doing it once. Here: the same key plus the same body returns the original result instead of creating another billable OpenAI request.

12. How does Redis idempotency prevent duplicate billing?

Hash the validated request. SET NX an in_progress record before calling OpenAI. Only the claimant may create work. Later requests replay, conflict, or see in-progress.

13. What happens when the provider call fails?

The in-progress record is deleted after the final failure so a later retry can claim the key. A permanent lock would punish the client for OpenAI being down.

14. What is the difference between retries and idempotency?

Retries are the server trying OpenAI again after a blip. Idempotency is the client sending the same request again. You need both.

15. Which failures should be retried?

Transient ones: connection errors, timeouts, provider 5xx, and provider 429 when a retry hint exists. Not auth, not validation, not unsupported models.

16. Why should authentication errors not be retried?

They will fail the same way. Retrying only adds latency and can look like an attack.

17. What is exponential backoff? Why jitter?

Each retry waits longer (0.1s, 0.2s, ...). Jitter adds a small random amount so many instances do not retry in lockstep after an outage.

18. What is a correlation ID? How is it different from an idempotency key?

A correlation ID traces one request across logs, errors, and jobs. An idempotency key prevents duplicate work. The first is for debugging. The second is for correctness and cost.

19. Do correlation IDs authorize a request?

No. Bearer auth proves the caller has the internal key. A correlation ID only names the request.

20. What is background mode?

The wrapper asks OpenAI to run the response asynchronously, returns 202 and a job id immediately, and later updates that job from a webhook.

21. Why use webhooks instead of polling OpenAI forever?

Polling OpenAI from every status request would add load, latency, and cost. Redis is the source of truth for the client. Webhooks update Redis when OpenAI is done.

22. Why can webhook events be duplicated?

Networks retry. OpenAI will resend if it is not sure you got the event. Dedup by event id in Redis makes that harmless.

23. How should out-of-order events be handled?

Never overwrite a terminal job. The finalize Lua script writes the new status only if the job is not already completed, failed, cancelled, incomplete, or expired.

24. Why keep processed webhook event IDs?

So a second delivery of the same event is a no-op across workers. The processed mark lasts three days, long enough to cover provider retries.

25. Why return an error instead of acknowledging a temporary webhook failure?

Success means “stop sending.” A 503 preserves OpenAI’s retry mechanism.

26. Why store a wrapper job instead of returning only the OpenAI response id?

The wrapper job keeps provider identifiers mostly internal and gives clients a stable status URL, expiry, correlation ID, and controlled state model.

27. Why isolate the OpenAI SDK behind a client adapter?

Routes should own the wrapper contract, not SDK types. One place to construct the client, one place to fake it in tests.

28. Why avoid logging prompts and outputs by default?

They can contain customer data. Correlation IDs plus event names are enough to debug most production issues.

29. How do mocked tests differ from live smoke tests?

The default suite fakes OpenAI and Redis behavior and must stay free. A live smoke test would use a real key, cost money, and should never run automatically.

30. What would need to change for multiple internal clients?

Per-client wrapper keys, per-client rate limits, and probably per-client allowlists and audit fields. Today there is one shared WRAPPER_API_KEY.

31. What would need to change for multi-region?

Redis would need to be regional or replicated with a clear consistency story. Idempotency and webhook claims only work if all instances share the same keyspace.

18. Words you must not mix up

Word	Means	Does not mean
Wrapper	Your internal service in front of OpenAI	A generic HTTP proxy that forwards everything
Correlation ID	A name tag for tracing one request	A password or a permission

Word	Means	Does not mean
Idempotency key	A client key that makes a retry return the original result	A request id you generate for logs
Retry	This service calling OpenAI again after a transient failure	The client sending the request again
Rate limit	This wrapper's Redis counter for the internal app	OpenAI's own rate limit (that becomes 503 here)
Live	The process is running	The process can do useful work
Ready	Required dependencies (Redis) answer	OpenAI is healthy right now
TTL	Redis will delete the key after this many seconds	The client's poll interval
Background response	OpenAI finishes after the HTTP request returns 202	A local worker queue inside this repo
Webhook	OpenAI calling you with a signed event	You polling OpenAI
Fail closed	If Redis is down, refuse the request (503)	Skip the check and let traffic through
Normalize	Return only the fields your contract promises	Return the raw OpenAI object

Last page checklist

- I can draw the architecture and name the six endpoints.
- I can say the middleware order and why it is that order.
- I can whiteboard a sync request including idempotency states.
- I can explain background create, poll codes, and webhook verify-then-claim.
- I can list the Redis key prefixes.
- I can map common failures to HTTP codes without looking.
- I can say what is not built yet without sounding defensive.

If you can do those seven things, you understand this project end to end. The files are only there to prove the story, not to replace it.